TASK 1 — Advanced Conditional Logic & Flow Design

Key Concept Definitions

if/else Statement vs Expression

An **if/else statement** executes a block of code depending on a condition.
An **if/else expression** returns a value and can be directly assigned to a variable.

Example — if/else Expression

```kotlin
val result = if (amount > 100) "High Expense" else "Low Expense"
```

Here, `if` returns a value.

1.2 when Statement vs Expression

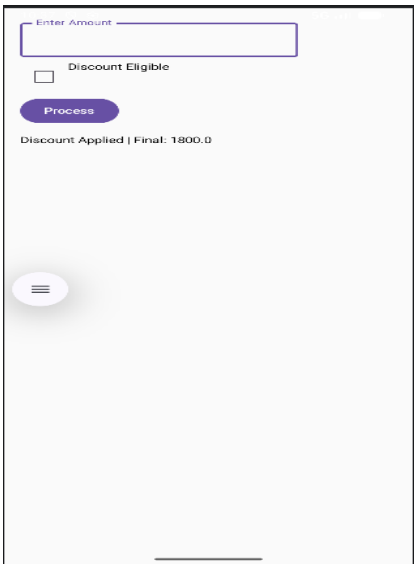
A **when statement** works like a switch statement.
A **when expression** returns a value and can be stored in a variable.

1.3 Use of `in`, `is`, and Multiple Branches in `when`

- `in` → Used to check ranges.
- `is` → Used to check type.
- Multiple values can share one branch.

Example

```kotlin
fun checkInput(input: Any): String {
    return when (input) {
        is String -> "It is a String"
        in 1..100 -> "Number in range"
        else -> "Unknown"
    }
}
```



App Feature — Smart Expense Validator

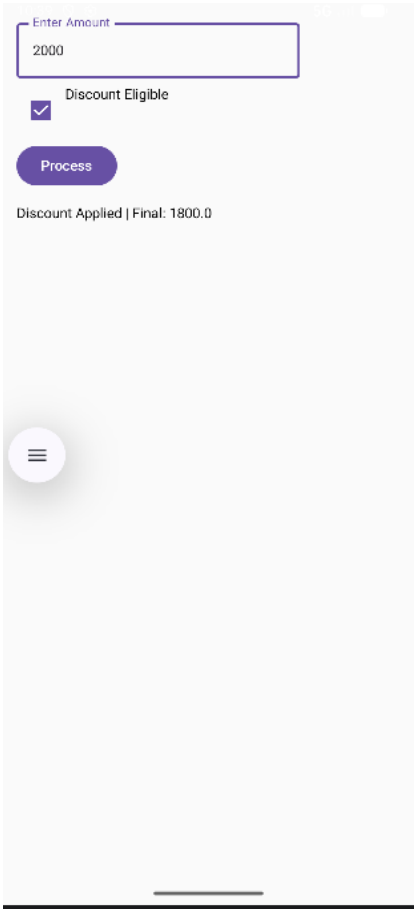
This feature validates an expense amount, categorizes it, and applies discount or surcharge based on conditions.

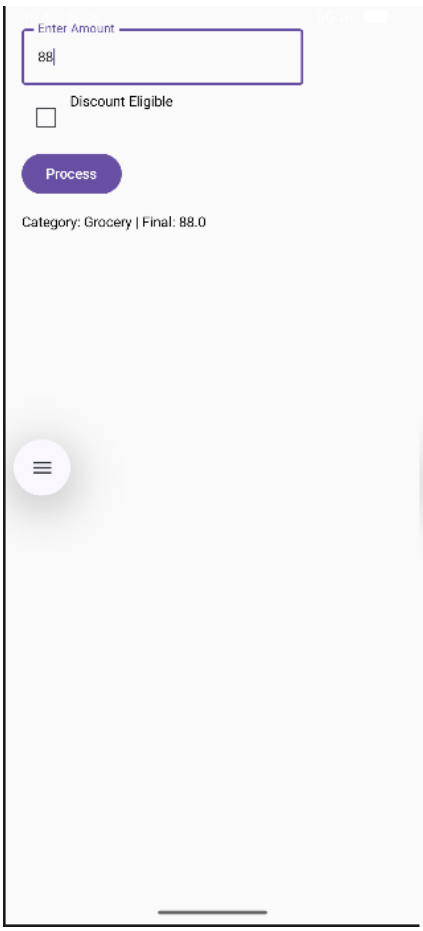
Decisions made:

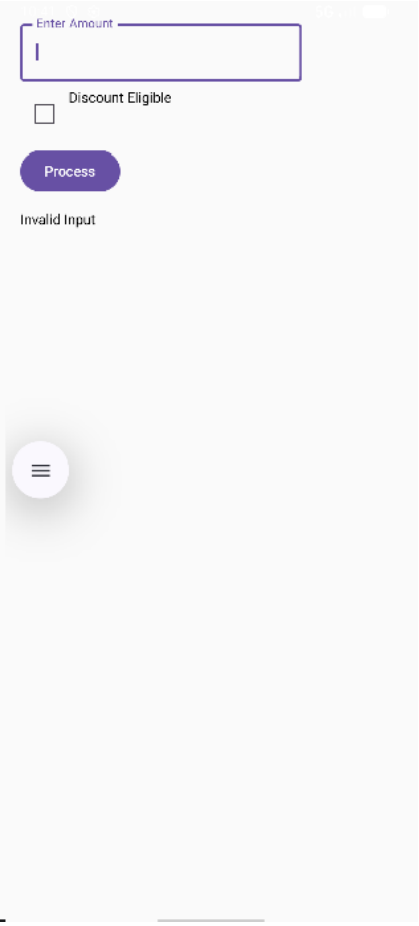
- If amount < 0 → Reject transaction
- If amount in 0..100 → Grocery category
- If amount > 100 and discountEligible → Apply 10% discount
- Else → Apply 5% surcharge

Decision Flow Table

Condition	Action	Code Construct
amount < 0	Reject	if
amount in 0..100	Grocery	when (in range)
amount > 100 AND discountEligible	Apply discount	when + condition
else	Apply surcharge	else branch







Reflection

Planning complex logic before coding improves clarity and reduces errors.Without structured decision flow, nested conditions become difficult to debug. Using expressions instead of deeply nested if/else improves readability. Clear branching improves maintainability and scalability of UI features.

TASK 2 — Nullability, Safe Calls & Robust Data Handling

1. Key Concepts

Nullable vs Non-Nullable Types

In Kotlin, a non-nullable type (String) cannot store null values, while a nullable type (String?) can store either a value or null.

Safe Call Operator (?.)

Used to safely access properties of a nullable object without causing a crash.

Example:

```kotlin
val length = nickname?.length
```

Elvis Operator (?:)

Provides a default value if the variable is null.

Example:

```kotlin
val name = user?.nickname ?: "Guest"
```

Non-Null Assertion (!!)

Forces a nullable value to be non-null. If null, it crashes the app.

Example:

```kotlin
val length = nickname!!.length
```

Data Handling Scenario

In this feature, the app fetches a User object from a remote API. Some fields like nickname, age, and photoUrl may be null because the user did not provide them. Nullability handling ensures the app does not crash and instead shows meaningful fallback values.

Null-Safe Logic Table

Field	Type	Nullable	Handling Strategy	UI Result
name	String	No	Direct use	Lakshmitha
nickname	String?	Yes	Elvis operator	Guest
age	Int?	Yes	Safe call + Elvis	N/A
photoUrl	String?	Yes	Null check	Placeholder

Reflection

Null safety is crucial in Android apps because network data may be incomplete or missing. If null values are not handled properly, apps crash due to NullPointerException. Using safe calls and the Elvis operator ensures stable and reliable UI behavior. Designing null-safe systems improves user experience and prevents runtime failures.

TASK 3 — Feature Implementation Strategy

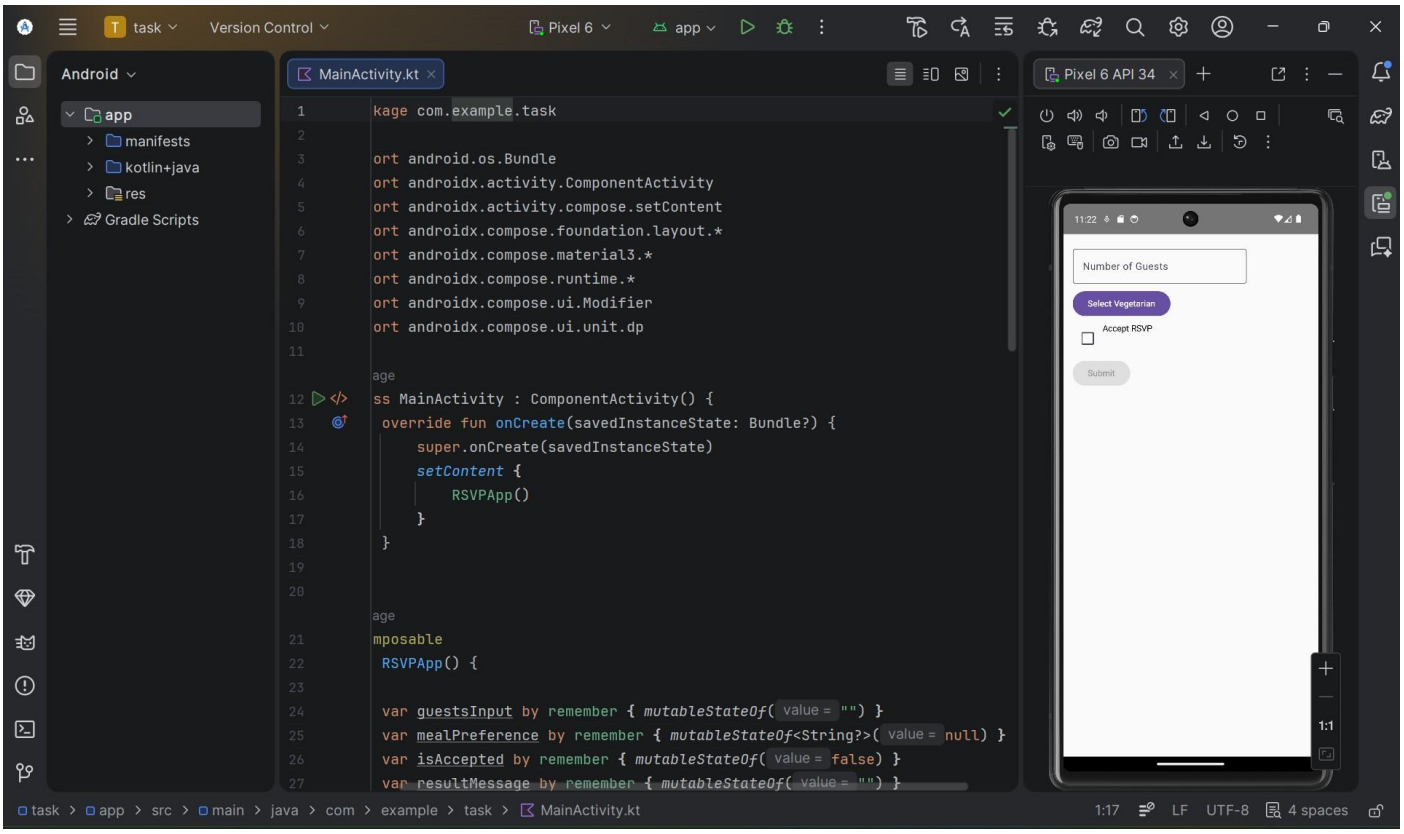
Feature Definition

Feature: Event RSVP Screen

This feature allows a user to enter the number of guests, select a meal preference, and choose whether to accept or decline the RSVP invitation.

The logic ensures:

- If the user accepts and guests > 0 → Submit is enabled.
- If the user declines → A “Thank you” message is shown.
- If meal preference is not selected → Default value “Standard” is used.



Data & State Model

Data Fields

Field Name	Type	Nullable
guests	Int?	Yes
mealPreference	String?	Yes
isAccepted	Boolean	No

Compose State Variables Used

```kotlin
var guestsInput by remember { mutableStateOf("") }
var mealPreference by remember { mutableStateOf<String?>(null) }
var isAccepted by remember { mutableStateOf(false) }
var resultMessage by remember { mutableStateOf("") }
```

State Transitions

- If `guestsInput` is blank → parsing returns null.
- If guests ≤ 0 → error message shown.
- If RSVP not accepted → Thank you message shown.
- If mealPreference is null → default "Standard" used.

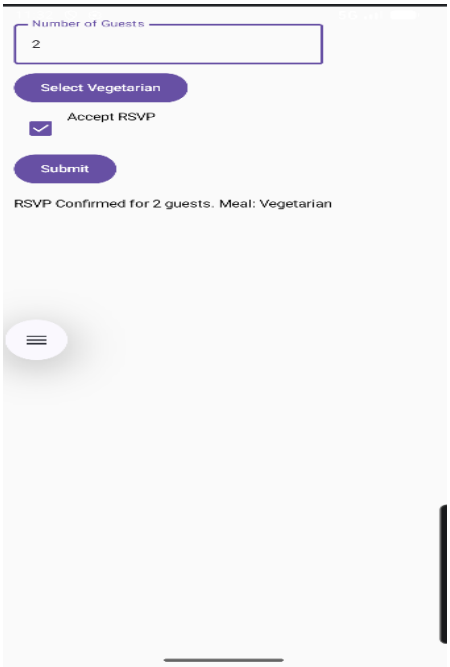
Conditional & Null-Safe Logic Used

This implementation uses:

- `toIntOrNull()` → Safe parsing
- Elvis operator `?:`
- if/else conditional branching
- Nullable type handling

UI Conditions Mapping

UI Element	Condition	Behavior
Submit Button	Enabled only if Accept checked	Prevents invalid submission
Guest Input	Blank or 0	Shows error
Meal Preference	Null	Defaults to Standard
Decline Option	Not accepted	Shows Thank You



Edge Case & Test Planning

Edge Case	Expected Outcome	Logic Handling
Guests blank	Error message	toIntOrNull() returns null
Guests = 0	Error message	guests <= 0 check
Meal not selected	Default Standard	Elvis operator
Rapid checkbox toggle	UI updates properly	Compose state recomposition

Reflection

This feature is more complex because it combines conditional logic with nullable input handling. Managing multiple UI states increased cognitive complexity. Safe parsing using `toIntOrNull()` prevented crashes from invalid input. Using default values with the Elvis operator simplified logic. Planning state transitions before coding made implementation structured and maintainable.